

Estructuras de Datos y Algoritmos

Tema 5: Recursión

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

Intuición

Un algoritmo recursivo resuelve un problema llamándose a sí mismo sobre una o más instancias más pequeñas del mismo problema.

La idea central no es repetir indefinidamente, sino reducir el problema hasta llegar a un caso que podamos resolver directamente.

Esquema básico

problema grande $\longrightarrow$ problemas más pequeños $\longrightarrow$ caso base

Estructura de un algoritmo recursivo

Todo algoritmo recursivo correcto debe dejar claras tres partes:

- **Caso base:** instancia que se resuelve directamente, sin hacer una llamada recursiva.
- **Caso recursivo:** reducción del problema original a una o más instancias del mismo tipo.
- **Progreso:** cada llamada recursiva debe acercarse al caso base.

Si falta el caso base, o si las llamadas recursivas no progresan, el algoritmo puede no terminar.

Recursión directa e indirecta

Podemos distinguir dos formas de recursividad:

Recursión directa

Una función se llama a sí misma.

$$f \longrightarrow f$$

Recursión indirecta

Una función llama a otra función que, directa o indirectamente, vuelve a llamar a la primera.

$$f \longrightarrow g \longrightarrow f$$

Ejemplo: factorial

El factorial de un entero $n \geq 0$ se define por:

$$n! = \begin{cases} 1, & n = 0, \\ n \cdot (n-1)!, & n > 0. \end{cases}$$

Esta definición ya es recursiva:

- El caso base es $0! = 1$.
- El caso recursivo expresa $n!$ usando $(n-1)!$.
- El progreso ocurre porque $n-1 < n$.

Ejemplo: factorial en Python

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

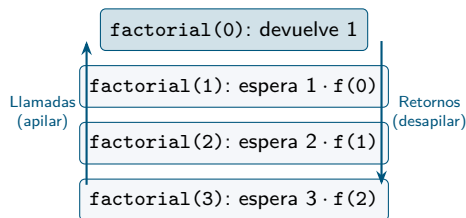
Lectura

Para calcular `factorial(n)`, primero necesito calcular `factorial(n - 1)`.

Ejemplo con $n = 4$:

$$\begin{aligned}4! &= 4 \cdot 3! \\ &= 4 \cdot 3 \cdot 2! \\ &= 4 \cdot 3 \cdot 2 \cdot 1! \\ &= 4 \cdot 3 \cdot 2 \cdot 1 \cdot 0! \\ &= 24.\end{aligned}$$

Pila de llamadas y memoria



Flujo de retornos: $1 \rightarrow 1 \rightarrow 2 \rightarrow 6$.

Marcos de activación:

Cada llamada suspende su ejecución y reserva memoria para parámetros, variables y dirección de retorno.

Stack Overflow: Si no hay caso base o n es muy grande, se agota la pila (RecursionError en Python).

Recursión e iteración

Una solución recursiva puede ser más clara, pero no necesariamente más eficiente.

Recursión	Iteración
Expresa naturalmente problemas definidos en términos de sí mismos.	Suele evitar el costo extra de las llamadas a función.
Es útil en árboles, búsqueda, divide y vencerás y definiciones inductivas.	Suele ser más directa cuando el problema es una repetición lineal.
Puede aumentar complejidad si se usa sin cuidado.	Puede requerir variables auxiliares que oculten la estructura matemática.

Fibonacci es un ejemplo clásico: la versión recursiva ingenua es clara, pero repite muchos cálculos.

Terminación y corrección

La recursión se conecta naturalmente con las ideas de corrección parcial y total.

Para una función recursiva queremos justificar dos cosas:

- **Corrección parcial:** si la función termina, entonces devuelve el resultado especificado.
- **Corrección total:** toda cadena de llamadas recursivas llega al caso base en un número finito de pasos.

$\text{corrección total} = \text{corrección parcial} + \text{terminación}.$

De código a recurrencias

```
def suma(n):  
    if n == 0:  
        return 0  
    return n + suma(n - 1)
```

Conteo en el modelo RAM:

- **Caso base ($n = 0$):**
Evaluar condición y retornar 0: $T(0) = c_1$.
- **Caso recursivo ($n > 0$):**
Evaluar, restar, sumar y retornar (c_2 pasos),
más la llamada recursiva:

$$T(n) = T(n - 1) + c_2.$$

Ecuación en recurrencia:

$$T(n) = \begin{cases} c_1, & n = 0, \\ T(n - 1) + c_2, & n > 0. \end{cases}$$

Desenrollado (expansión):

$$\begin{aligned} T(n) &= T(n - 1) + c_2 \\ &= T(n - 2) + 2c_2 \\ &\vdots \\ &= T(0) + n \cdot c_2 \\ &= c_1 + c_2 n \in \Theta(n). \end{aligned}$$

La recurrencia formaliza el costo ante la autorreferencia.

Divide y vencerás

Divide y vencerás es una estrategia de diseño algorítmico que resuelve un problema descomponiéndolo recursivamente:

- ➊ **Dividir:** separar el problema en subproblemas más pequeños del mismo tipo.
- ➋ **Conquistar:** resolver los subproblemas de forma recursiva (o directa si son pequeños).
- ➌ **Combinar:** unir las soluciones parciales para formar la solución global.

Forma común de la recurrencia

$$T(n) = a T\left(\frac{n}{b}\right) + f(n), \quad T(1) = \Theta(1).$$

- a : número de subproblemas en cada división ($a \geq 1$).
- b : factor de reducción de tamaño ($b > 1$).
- $f(n)$: costo no recursivo de **dividir** y **combinar**.

Ejemplo: potenciación entera

Problema

Dados $x \in \mathbb{R}$ y un entero $n \geq 0$, calcular la potencia x^n .

Enfoque iterativo ingenuo:

$$x^n = \underbrace{x \cdot x \cdot \dots \cdot x}_{n \text{ veces}}$$

Requiere $n - 1$ multiplicaciones:

$$T(n) \in \Theta(n).$$

Idea de Divide y Vencerás: Aprovechar propiedades algebraicas:

$$x^n = \begin{cases} 1, & n = 0, \\ \left(x^{n/2}\right)^2, & n \text{ par}, \\ x \cdot \left(x^{(n-1)/2}\right)^2, & n \text{ impar}. \end{cases}$$

Clave: Para calcular x^n , ¡basta con calcular $x^{\lfloor n/2 \rfloor}$ **una sola vez!**

Potenciación rápida: código y costo

```
def potencia(x, n):  
    if n == 0:  
        return 1  
    mitad = potencia(x, n // 2)  
    if n % 2 == 0:  
        return mitad * mitad  
    else:  
        return x * mitad * mitad
```

Al guardar *mitad*, realizamos **una sola** llamada recursiva con $n/2$.

Estructura Divide y Vencerás:

- **Dividir:** calcular $n//2$ en tiempo $\Theta(1)$.
- **Conquistar:** $a = 1$ subproblema de tamaño $n/2$ ($b = 2$).
- **Combinar:** a lo sumo 2 multiplicaciones en tiempo $\Theta(1)$.

Recurrencia resultante:

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1)$$

$$\implies T(n) \in \Theta(\log n).$$

Algunos métodos para resolver recurrencias son:

- **Sustitución:** se propone una cota y se demuestra por inducción matemática.
- **Árbol recursivo:** se representa el costo por niveles y se suman los costos de todos los niveles.
- **Método maestro:** se aplica a recurrencias de la forma $T(n) = aT(n/b) + f(n)$.
- **Akra-Bazzi:** generaliza el método maestro para recurrencias más flexibles.

En esta primera aproximación desarrollaremos sustitución y árbol recursivo.

Método de sustitución

El método de sustitución tiene dos pasos:

- 1 **Proponer** la forma de la solución.
- 2 **Probar** la cota por inducción matemática.

La dificultad principal es elegir una buena conjetura.

Idea

Si sospechamos que $T(n) \in \Theta(g(n))$, intentamos demostrar:

$$c_1g(n) \leq T(n) \leq c_2g(n)$$

para constantes positivas c_1, c_2 y para n suficientemente grande.

Sustitución: ejemplo 1

Considere:

$$T(1) = d, \quad T(n) = T(n-1) + cn, \quad c, d > 0.$$

Conjeturamos que $T(n) \leq an^2$ para alguna constante $a > 0$.

- **Paso inductivo:** Suponiendo $T(n-1) \leq a(n-1)^2$:

$$T(n) \leq a(n-1)^2 + cn = an^2 - (2a - c)n + a.$$

Para que esto sea $\leq an^2$, requerimos $-(2a - c)n + a \leq 0$. Si elegimos $a \geq c$, entonces para todo $n \geq 1$:

$$-(2a - c)n + a \leq -an + a = -a(n-1) \leq 0.$$

- **Caso base ($n = 1$):**

$$T(1) = d \leq a(1)^2 \iff a \geq d.$$

Conclusión: Fijando $a \geq \max(c, d)$, la cota se cumple para todo $n \geq 1$, por lo que $T(n) \in \mathcal{O}(n^2)$. Con una cota inferior análoga, $T(n) \in \Theta(n^2)$.

Sustitución: ejemplo 2

Considere:

$$T(1) = d, \quad T(n) = T\left(\frac{n}{2}\right) + cn, \quad c, d > 0.$$

Conjeturamos que $T(n) \leq an$ para alguna constante $a > 0$.

- **Paso inductivo:** Suponiendo $T(n/2) \leq a(n/2)$:

$$T(n) \leq a\frac{n}{2} + cn = \left(\frac{a}{2} + c\right)n.$$

Para que sea $\leq an$, requerimos $\frac{a}{2} + c \leq a \iff a \geq 2c$.

- **Caso base ($n = 1$):**

$$T(1) = d \leq a(1) \iff a \geq d.$$

Conclusión: Eligiendo $a \geq \max(2c, d)$, se cumple $T(n) \leq an$ para todo $n \geq 1$. Como además $T(n) \geq cn$, concluimos que $T(n) \in \Theta(n)$.

Árbol de recursión

Cada nodo representa una llamada. Su etiqueta indica el trabajo realizado **fuera de las llamadas recursivas**.

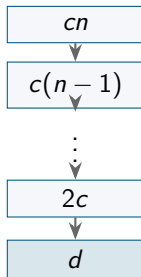
- **Raíz:** la llamada inicial.
- **Ramas:** las llamadas a subproblemas.
- **Hojas:** las llamadas que resuelven un caso base.
- **Nivel:** llamadas a la misma profundidad; la raíz está en el nivel 0.

$$\text{costo total} = \sum_{\text{niveles internos}} \text{costo del nivel} + \text{costo de las hojas}.$$

El árbol ayuda a obtener una cota que después podemos demostrar por sustitución.

Ejemplo: una rama

$$T(n) = T(n-1) + cn, \quad T(1) = d, \quad c, d > 0.$$



Hay $n - 1$ niveles internos y una hoja.

$$\begin{aligned} T(n) &= cn + c(n-1) + \cdots + 2c + d \\ &= c \sum_{j=2}^n j + d \\ &= c \left(\frac{n(n+1)}{2} - 1 \right) + d. \end{aligned}$$

Por tanto,

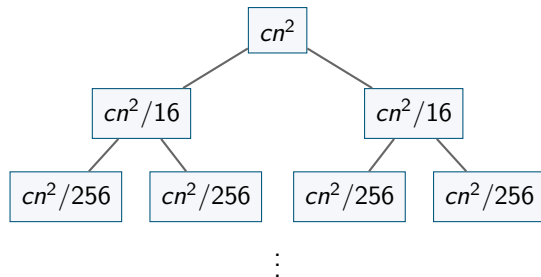
$$T(n) \in \Theta(n^2).$$

Coincide con el resultado del método de sustitución.

Ejemplo: dos ramas

$$T(n) = 2T(n/4) + cn^2, \quad T(1) = d, \quad c, d > 0.$$

Suponemos $n = 4^h$. Estos son los primeros niveles internos:



Nivel interno i , con $0 \leq i < h$

Nodos	2^i
Tamaño por nodo	$n/4^i$
Costo por nodo	$cn^2/16^i$
Costo del nivel	$cn^2/8^i$

Altura y hojas

Retomamos $T(n) = 2T(n/4) + cn^2$, con $n = 4^h$ y $T(1) = d$.

Cuándo termina la división

El tamaño del subproblema llega a 1:

$$\frac{n}{4^h} = 1$$

$$4^h = n$$

$$h = \log_4 n.$$

La raíz está en el nivel 0 y las hojas en el nivel h : hay $h + 1$ niveles en total.

Cuántas hojas aparecen

En cada división se generan dos llamadas:

$$\begin{aligned} 2^h &= 2^{\log_4 n} \\ &= \left(4^{1/2}\right)^{\log_4 n} \\ &= n^{1/2} = \sqrt{n}. \end{aligned}$$

Cada hoja cuesta d . Su costo conjunto es

$$\boxed{d\sqrt{n}.$$

Costo total

Sea $h = \log_4 n$. Sumamos los niveles internos y después las hojas:

$$T(n) = cn^2 \sum_{i=0}^{h-1} \left(\frac{1}{8}\right)^i + d\sqrt{n}.$$

Cota superior. La serie geométrica permite acotar la suma:

$$\begin{aligned} T(n) &\leq cn^2 \sum_{i=0}^{\infty} \left(\frac{1}{8}\right)^i + d\sqrt{n} \\ &= \frac{8}{7}cn^2 + d\sqrt{n} \leq \left(\frac{8}{7}c + d\right)n^2. \end{aligned}$$

Cota inferior. Para $n = 4^h \geq 4$, la raíz por sí sola cuesta cn^2 . Todos los demás costos son positivos. Por tanto,

$$cn^2 \leq T(n) \leq \left(\frac{8}{7}c + d\right)n^2, \quad n = 4^h \geq 4.$$

$T(n) \in \Theta(n^2).$

Teorema maestro

Teorema maestro

Sean $a \geq 1$, $b > 1$ constantes y $f(n) \geq 0$. Consideremos

$$T(n) = aT(n/b) + f(n), \quad T(1) = d > 0, \quad p = \log_b a.$$

- ❶ Si $f(n) \in \mathcal{O}(n^{p-\varepsilon})$ para alguna constante $\varepsilon > 0$, entonces $T(n) \in \Theta(n^p)$.
- ❷ Si $f(n) \in \Theta(n^p)$, entonces $T(n) \in \Theta(n^p \log n)$.
- ❸ Si $f(n) \in \Omega(n^{p+\varepsilon})$ para alguna constante $\varepsilon > 0$, y además

$$af(n/b) \leq \rho f(n) \quad \text{para alguna constante } 0 < \rho < 1$$

para n suficientemente grande, entonces $T(n) \in \Theta(f(n))$.

Demostracion: pag 107 Introduction to algorithms 4ed Cormen T., Leiserson C., Rivest R. and Stein C.

Ejemplo: caso 1

$$T(n) = 4T(n/2) + n, \quad T(1) = d > 0.$$

Parámetros

$$a = 4, \quad b = 2, \quad f(n) = n.$$

$$p = \log_2 4 = 2.$$

Comparamos $f(n)$ con n^2 .

Comparación

$$f(n) = n \in \mathcal{O}(n^{2-1}).$$

Se cumple el caso 1 con $\varepsilon = 1$

Conclusión

$$T(n) \in \Theta(n^2).$$

Ejemplo: caso 2

$$T(n) = 2T(n/2) + n, \quad T(1) = d > 0.$$

Parámetros

$$a = 2, \quad b = 2, \quad f(n) = n.$$

$$p = \log_2 2 = 1.$$

Comparamos $f(n)$ con n .

Comparación

$$f(n) = n \in \Theta(n^1).$$

Se cumple el caso 2

Conclusión

$$T(n) \in \Theta(n \log n).$$

Ejemplo: caso 3

Retomamos el ejemplo resuelto mediante el árbol:

$$T(n) = 2T(n/4) + cn^2, \quad T(1) = d, \quad c, d > 0.$$

Parámetros

$$a = 2, \quad b = 4, \quad f(n) = cn^2.$$

$$p = \log_4 2 = \frac{1}{2}.$$

Comparamos $f(n)$ con $n^{1/2}$.

Comparación y regularidad

$$f(n) \in \Omega(n^{1/2+3/2}), \quad \varepsilon = \frac{3}{2}.$$

$$2f(n/4) = 2c(n/4)^2 = \frac{1}{8}f(n).$$

Se cumple el caso 3 con $\rho = 1/8 < 1$.

Conclusión

$$T(n) \in \Theta(n^2).$$

Algunas formas de recurrencias que no aplican al método maestro son:

Recurrencia

Por qué no aplica la versión presentada

$$T(n) = T(n-1) + n$$

La reducción es aditiva, no por un factor fijo.

$$T(n) = 2^n T(n/2) + n$$

El coeficiente de la llamada depende de n .

$$T(n) = T(n/3) + T(2n/3) + cn$$

Los subproblemas tienen tamaños distintos.

Sean $n \geq 1, m \geq 1, a_i > 0, b_i > 1, d > 0$:

$$T(n) = \begin{cases} d, & 1 \leq n \leq n_0, \\ \sum_{i=1}^m a_i T(n/b_i) + f(n), & n > n_0. \end{cases}$$

Condición de crecimiento. Existen constantes $C_1, C_2 > 0$ tales que, para todo i y $n > n_0$,

$$C_1 f(n) \leq f(u) \leq C_2 f(n), \quad n/b_i \leq u \leq n.$$

Teorema de Akra–Bazzi

Teorema de Akra–Bazzi

Bajo las hipótesis anteriores, existe un único $p \in \mathbb{R}$ que satisface

$$\sum_{i=1}^m \frac{a_i}{b_i^p} = 1.$$

Entonces

$$T(n) \in \Theta\left(n^p \left(1 + \int_1^n \frac{f(u)}{u^{p+1}} du\right)\right).$$

- p es una constante determinada por los coeficientes y los factores de reducción; no depende de n .
- u es la variable de integración. Usamos la misma función de costo no recursivo g .

Referencia: T. Leighton, *Notes on Better Master Theorems*, teorema 1.

Ejemplo: división desigual

Esta recurrencia no tiene la forma del teorema maestro:

$$T(n) = T(n/3) + T(2n/3) + cn, \quad n > 3, \quad c > 0.$$

Tomamos $T(n) = d > 0$ para $1 \leq n \leq 3$. El costo $f(u) = cu$ satisface la condición de crecimiento.

1. Encontrar p . Con $a_1 = a_2 = 1$, $b_1 = 3$ y $b_2 = 3/2$:

$$(1/3)^p + (2/3)^p = 1 \implies p = 1.$$

2. Calcular la integral.

$$\int_1^n \frac{cu}{u^{1+1}} du = c \int_1^n \frac{1}{u} du = c \ln n.$$

3. Sustituir.

$$T(n) \in \Theta(n(1 + c \ln n)) = \boxed{\Theta(n \log n)}.$$

$\ln n$ y $\log n$, en cualquier base fija mayor que 1, difieren por un factor constante.

Búsqueda con retroceso

Usaremos llamadas recursivas para explorar decisiones y construir soluciones.

Definición: búsqueda con retroceso

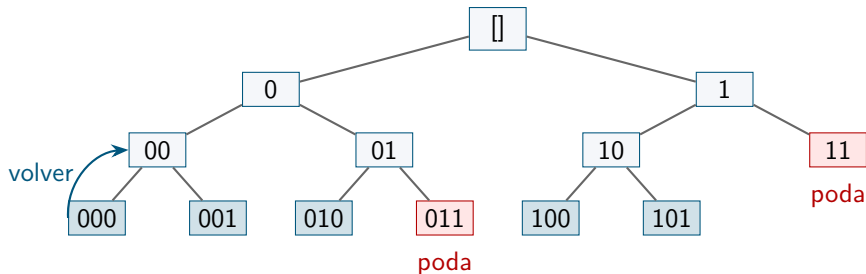
La búsqueda con retroceso (*backtracking*) construye soluciones de forma incremental, explora alternativas y descarta estados parciales que no pueden conducir a una solución válida.

- **Construir:** elegir una alternativa para extender la solución parcial.
- **Podar:** descartar una posibilidad que ya contradice las restricciones.
- **Retroceder:** restaurar el estado anterior para probar otra alternativa, también después de encontrar una solución.

Elegir $\longrightarrow$ **explorar** $\longrightarrow$ **deshacer**.

Ejemplo: decisiones

Construir secuencias binarias de longitud 3 sin dos unos consecutivos. Cada nodo muestra un prefijo de la secuencia.



Después de registrar 000, retiramos el último 0 y probamos 001.

Soluciones: 000, 001, 010, 100, 101. El prefijo 11 se descarta sin generar 110 ni 111.

Estados y poda

Un estado parcial es

$$X = (x_1, \dots, x_k), \quad 0 \leq k \leq n, \quad x_i \in S_i,$$

donde los conjuntos S_i son finitos. En el ejemplo, $S_i = \{0, 1\}$.

Admisibilidad de $P(X)$:

- Si $P(X)$ es falso, ninguna extensión de X debe ser una solución válida. Podemos podar sin perder soluciones.
- Si $P(X)$ es verdadero y $k < n$, todavía hay decisiones por explorar; esto no garantiza encontrar una solución.
- Para $k = n$, $P(X)$ comprueba que la solución completa cumple las restricciones.

Sin poda, el número de candidatos completos es

$$\prod_{i=1}^n |S_i|; \quad \text{si todos tienen } m \text{ elementos, son } m^n.$$

La poda puede ahorrar trabajo, pero no garantiza evitar un peor caso exponencial.

Algoritmo Backtracking

Usamos una lista mutable X . Iniciamos con $\text{Buscar}([])$.

Procedimiento $\text{Buscar}(X)$

Si $|X| = n$:

 Registrar una copia de X

 Retornar

Para cada $c \in S_{|X|+1}$:

 Si $P(X \text{ seguido de } c)$:

 Añadir c al final de X

$\text{Buscar}(X)$

 Retirar el último elemento de X

En el ejemplo, P rechaza prefijos con dos unos consecutivos.

Restauración del estado: después de explorar una alternativa, X queda como estaba antes de elegirla.

Terminación: cada llamada aceptada completa una posición; quedan finitas posiciones y cada una tiene finitas alternativas.